```
    the page rendered by this template.
  </p>
```

Then, in the layout, use `yield :sidebar` to include this block in the page's sidebar:

```
<!DOCTYPE .... >
<html>
  <body>
    <div class="sidebar">
      <p>
        Regular sidebar stuff
      </p>
      <div class="page-specific-sidebar">
»       <%= yield :sidebar %>
      </div>
    </div>
  </body>
</html>
```

This same technique can be used to add page-specific JavaScript functions into the `<head>` section of a layout, create specialized menu bars, and so on.

Partial-Page Templates

Web applications commonly display information about the same application object or objects on multiple pages. A shopping cart might display an order line item on the shopping cart page and again on the order summary page. A blog application might display the contents of an article on the main index page and again at the top of a page soliciting comments. Typically this would involve copying snippets of code between the different template pages.

Rails, however, eliminates this duplication with the *partial-page templates* (more frequently called *partials*). You can think of a partial as a kind of subroutine. You invoke it one or more times from within another template, potentially passing it objects to render as parameters. When the partial template finishes rendering, it returns control to the calling template.

Internally, a partial template looks like any other template. Externally, there's a slight difference. The name of the file containing the template code must start with an underscore character, differentiating the source of partial templates from